

Smart Receipt (eSal) — Mobile API Reference

AMAD Hackathon · Backend for the Swift mobile app · Prepared 2026-07-12

Base URL: `https://<your-render-app>.onrender.com` — replace with the actual Render deployment URL you've been testing against in Postman. All paths below are relative to this base URL.

General Conventions

Authentication

Every endpoint marked **JWT REQUIRED** needs this header on the request:

```
Authorization: Bearer <accessToken>
```

The `accessToken` comes from `POST /auth/login`. There is no separate "get current user id" call needed — the token is self-contained; the backend extracts the user's identity from it. The app never sends a user id anywhere itself.

Error response shape

All errors (validation, auth, not found, etc.) come back as JSON in Nest's standard shape:

```
{
  "statusCode": 404,
  "message": "Receipt not found",
  "error": "Not Found"
}
```

`message` can also be an **array of strings** specifically for request-validation failures (missing/invalid fields), e.g. `["email must be an email", "password must be longer than or equal to 8 characters"]`.

Important — Decimal fields serialize as JSON strings, not numbers.

Every money/quantity field (`subtotal` , `tax` , `serviceCharge` , `discount` , `total` , `quantity` , `unitPrice` , `totalPrice` , `totalSpendingThisMonth` , `totalSpending` , `category total`) is a Prisma `Decimal` under the hood. Decimals serialize to JSON as **quoted strings** (e.g. `"total": "49.90"`), not bare numbers — this is deliberate on Prisma's part, to avoid floating-point precision loss on money values. Swift's default `JSONDecoder` will **fail** to decode a quoted string directly into a `Double` / `Decimal` property — you'll need to either decode these fields as `String` and convert, or use a custom decoding strategy. The one exception: when a sum has *no data to sum* (e.g. no spending this month yet), the backend currently returns the plain number `0` instead of a Decimal string — so these specific fields can arrive as *either* a string or a number. Decode defensively.

Dates (`createdAt` , `updatedAt` , `warrantyEndDate`) are standard ISO-8601 UTC strings, e.g. `"2026-07-11T18:21:29.123Z"` .

HTTP status codes on `POST /auth/signup` and `POST /auth/login` : both currently return **201** on success (NestJS's default for `@Post()` routes when no status override is set), not 200. Check for the 2xx range rather than hardcoding 200 for these two.

Table of Contents

1. `POST /auth/signup`
 2. `POST /auth/login`
 3. `GET /users/me`
 4. `GET /receipts`
 5. `GET /receipts/:id`
 6. `GET /dashboard`
 7. `GET /insights`
 8. `GET /passes/wallet`
 9. Shop Owner Endpoints
 10. Not Included In This Doc
 11. Known Gaps / Open Items
-

1. Create Account

POST /auth/signup

NO AUTH

Request body

Field	Type	Required	Notes
email	string	yes	Must be a valid email; must be unique account-wide
password	string	yes	Minimum 8 characters
name	string	yes	Full name
phone	string	yes	
accountType	"CUSTOMER" "SHOP"	yes	Mobile app will always send "CUSTOMER"
storeName	string	only if accountType is "SHOP"	Not applicable to the customer app

```
{
  "email": "sara@example.com",
  "password": "hunter22",
  "name": "Sara Al-Otaibi",
  "phone": "+966500000000",
  "accountType": "CUSTOMER"
}
```

Success response — 201

Returns the full created user record (password always excluded):

```
{
  "id": "b2b6b7b0-....",
  "name": "Sara Al-Otaibi",
  "email": "sara@example.com",
  "phone": "+966500000000",
  "role": "USER",
  "accountType": "CUSTOMER",
  "walletToken": "6f0a1e2c-....",
  "createdAt": "2026-07-12T09:00:00.000Z",
  "updatedAt": "2026-07-12T09:00:00.000Z",
  "storeId": null
}
```

Note: this is currently the *only* response in the whole API that returns walletToken — see [Known Gaps](#) below.

Error responses

Status	Message	When
--------	---------	------

409	"An Account with this email already exists." (typo is in the real API response — match on status code, not string)	Email already registered
400	Array of validation messages	Missing/invalid fields, password < 8 chars, invalid accountType , extra unrecognized fields in the body
500	Generic error	Unexpected DB failure other than the unique-email conflict

2. Login

POST /auth/login

NO AUTH

Request body

```
{
  "email": "sara@example.com",
  "password": "hunter22"
}
```

Success response — 201

```
{
  "accessToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
}
```

Store this token and attach it as `Authorization: Bearer <accessToken>` on every subsequent authenticated request.

Error responses

Status	Message	When
401	"Invalid Credentials"	Email not found or wrong password — same message for both, deliberately, so the client can't probe which emails are registered
400	Array of validation messages	Missing/malformed email or password

3. Current User Profile

GET /users/me

JWT REQUIRED

No path or query parameters. Used for the Profile screen.

Success response — 200

```
{
  "id": "b2b6b7b0-....",
  "name": "Sara Al-Otaibi",
  "email": "sara@example.com",
  "phone": "+966500000000",
  "accountType": "CUSTOMER",
  "createdAt": "2026-07-12T09:00:00.000Z"
}
```

Deliberately does **not** include `role` , `walletToken` , `updatedAt` , or `storeId` .

Error responses

Status	Message	When
401	"Unauthorized"	Missing, malformed, or expired JWT
404	"User not found"	Token is valid but the user no longer exists (shouldn't normally happen)

4. Receipts List

GET /receipts

JWT REQUIRED

Powers the Receipts screen (list + filters + search). Only ever returns receipts claimed by the logged-in customer.

Query parameters (all optional)

Param	Type	Values	Effect
category	string	One of the 10 fixed category names, or "All"	Only receipts containing at least one line item in that category
range	string	"thisWeek" "lastMonth"	Filters by createdAt
search	string	free text	Case-insensitive substring match against store name OR any line item name
page	integer	≥ 1, default 1	Pagination
pageSize	integer	≥ 1, default 20	Pagination

The 10 fixed category names: Food & Dining , Coffee & Beverages , Groceries , Fashion & Apparel , Electronics , Books & Stationery , Transport , Health & Beauty , Entertainment , Other .

Any query param not listed above will cause a 400 — the backend rejects unrecognized params rather than silently ignoring them.

Example requests

```
GET /receipts
GET /receipts?category=Electronics
GET /receipts?range=thisWeek
GET /receipts?search=monir
GET /receipts?category=Groceries&range=lastMonth&page=2&pageSize=10
```

Success response — 200

```
{
  "data": [
    {
      "id": "c8f2....",
      "subtotal": "199.00",
      "tax": "29.85",
      "serviceCharge": null,
      "discount": null,
      "total": "228.85",
      "paymentMethod": "CARD",
      "invoiceNo": "INV-10234",
      "status": "ASSIGNED",
      "createdAt": "2026-07-10T14:22:00.000Z",
      "device": {
        "store": {
          "name": "Monir Electronics",
          "logoUrl": null,
          "taxId": null
        }
      },
    },
    "lineItems": [
      {
        "id": "9a1c....",
        "name": "USB-C Cable 2m",
        "quantity": "1",
        "unitPrice": "199.00",
        "totalPrice": "199.00",
        "warrantyEndDate": null,
        "category": { "name": "Electronics" }
      }
    ]
  },
  "pagination": {
    "page": 1,
    "pageSize": 20,
    "total": 3,
    "totalPages": 1
  }
}
```

An empty result set is valid, not an error: `"data": []` , `"pagination": { "total": 0, ... }` .

Error responses

Status	When
401	Missing/invalid JWT
400	Invalid <code>range</code> value, non-integer/<1 <code>page</code> / <code>pageSize</code> , or an unrecognized query param

5. Receipt Detail

GET /receipts/:id

JWT REQUIRED

Powers the Receipt Details screen. `:id` is the receipt's UUID (from a list item's `id`).

Success response — 200

```
{
  "id": "c8f2....",
  "subtotal": "199.00",
  "tax": "29.85",
  "serviceCharge": null,
  "discount": null,
  "total": "228.85",
  "paymentMethod": "CARD",
  "invoiceNo": "INV-10234",
  "status": "ASSIGNED",
  "createdAt": "2026-07-10T14:22:00.000Z",
  "device": {
    "store": { "name": "Monir Electronics", "logoUrl": null, "taxId": null }
  },
  "lineItems": [
    {
      "id": "9a1c....",
      "name": "USB-C Cable 2m",
      "quantity": "1",
      "unitPrice": "199.00",
      "totalPrice": "199.00",
      "warrantyEndDate": null,
      "category": { "name": "Electronics" }
    }
  ]
}
```

The barcode shown on this screen for the cashier is generated client-side from `invoiceNo` — the backend does not generate any barcode image.

Error responses

Status	Message	When
404	"Receipt not found"	<code>:id</code> doesn't exist at all
403	"This receipt does not belong to you"	Receipt exists but is unclaimed or claimed by a different customer
401	"Unauthorized"	Missing/invalid JWT

6. Dashboard

GET /dashboard

JWT REQUIRED

Single bundled call for the Dashboard home screen — first name, this-month spending, nearest warranty deadline, the 3 most recent receipts, and the offers card.

Success response — 200

```
{
  "firstName": "Sara",
  "totalSpendingThisMonth": "228.85",
  "nearestWarranty": {
    "lineItemId": "9a1c...",
    "itemName": "USB-C Cable 2m",
    "warrantyEndDate": "2027-07-10T00:00:00.000Z",
    "receiptId": "c8f2...",
    "storeName": "Monir Electronics",
    "storeLogoUrl": null
  },
  "recentReceipts": [
    { "...same shape as a /receipts list item...": true }
  ],
  "smartOffers": [
    {
      "title": "Extended Warranty Available",
      "description": "Protect your recent electronics purchase with an extended warranty plan.",
      "imageUrl": null
    },
    {
      "title": "10% Cashback at Partner Stores",
      "description": "Earn cashback when you shop at participating Smart Receipt partner stores",
      "imageUrl": null
    }
  ]
}
```

Field	Notes
firstName	First whitespace-separated token of the user's name field
totalSpendingThisMonth	Sum of total across all receipts claimed this calendar month. Plain 0 (number) if no receipts yet this month
nearestWarranty	null if the user has no line item with a future warrantyEndDate — this is a normal, expected state, not an error
recentReceipts	Up to 3, newest first — same object shape as the /receipts list
smartOffers	Fully hardcoded placeholder content — see Known Gaps

Error responses

Status	Message	When
401	"Unauthorized"	Missing/invalid JWT
404	"User not found"	Token valid but user no longer exists

7. Insights

GET /insights

JWT REQUIRED

Powers the Insights screen — total spend plus a per-category breakdown for a selectable time range.

Query parameters

Param	Values	Default
range	"thisMonth" "lastMonth" "3months"	"thisMonth" if omitted

Note the different vocabulary from the Receipts screen: Insights uses `thisMonth` / `lastMonth` / `3months` , while Receipts uses `thisWeek` / `lastMonth` . Don't reuse one range enum for both screens in the app.

Example requests

```
GET /insights
GET /insights?range=lastMonth
GET /insights?range=3months
```

Success response — 200

```
{
  "range": "thisMonth",
  "totalSpending": "560.40",
  "categories": [
    { "category": "Electronics", "total": "228.85" },
    { "category": "Groceries", "total": "331.55" }
  ]
}
```

`categories` is sorted by `total` descending. A category with no spending in range is simply absent from the array (not returned as zero). If a line item somehow has no assigned category, it's grouped under `"Uncategorized"` . Empty range → `"totalSpending": 0` , `"categories": []` .

Error responses

Status	When
401	Missing/invalid JWT
400	<code>range</code> is present but not one of the 3 allowed values

8. Apple Wallet Pass

GET /passes/wallet

JWT REQUIRED

Returns a signed `.pkpass` file for the logged-in user — this is the customer's membership card / wallet-token carrier, and is **how the app hands the customer's claim QR code to Apple Wallet**. The raw `walletToken` value never needs to be exposed to the app as JSON anywhere; it's baked directly into the pass's barcode by the backend.

Response — 200

Header	Value
Content-Type	application/vnd.apple.pkpass
Content-Disposition	attachment; filename="esal.pkpass"

Body is the raw binary `.pkpass` file (a signed zip archive). On iOS, fetch it as `Data`, then:

```
let pass = try PKPass(data: passData)
let vc = PKAddPassesViewController(pass: pass)
present(vc, animated: true)
```

The pass contains a `PKBarcodeFormatQR` barcode encoding the user's `walletToken`, their name, "Member Since" date, and account type. This is what the customer shows at checkout for the store's Raspberry Pi (GM65 scanner) to read and claim the receipt against their account.

Error responses

Status	Message	When
401	"Unauthorized"	Missing/invalid JWT
404	"User Doesn't Have Wallet Token"	User's <code>walletToken</code> is null (shouldn't normally happen — it's set automatically at account creation)
409	"Shop Owner Not Linked To A Store"	Only relevant for <code>SHOP</code> -type accounts, not the customer app's flow

Shop Owner Endpoints

Requires a fresh login. These 4 endpoints rely on `accountType` and `storeId` now being embedded in the JWT at login time. Any `accessToken` obtained *before* this change was deployed will not carry those claims and will be rejected with 403 by every endpoint below — call `POST /auth/login` again to get a token that works with them. This does not affect any of the customer endpoints above, which never read those claims.

All 4 endpoints below share the same guard stack: **JWT REQUIRED** **and** the account must be `accountType: "SHOP"`. A `CUSTOMER` token calling any of these gets a 403, not empty data.

Status	Message	When (applies to all 4 endpoints below)
401	"Unauthorized"	Missing/invalid JWT
403	"This endpoint is only available to shop accounts"	Token belongs to a CUSTOMER account
403	"Shop account is not linked to a store"	Defensive case — shouldn't happen in practice, since a store is created atomically at signup for SHOP accounts

Privacy by design: none of these endpoints ever return the identity (name/email/phone) of the customer who claimed a receipt — only the receipt's status ("PENDING" = unclaimed, "ASSIGNED" = claimed by some customer). This was a deliberate choice, not an oversight.

GET /shop/receipts

Query parameters (all optional)

Param	Type	Values	Effect
invoiceNo	string	free text	Case-insensitive partial match — the primary way a shop owner looks up a specific transaction
category	string	one of the 10 fixed category names, or "All"	Only receipts containing at least one line item in that category
range	string	"thisWeek" "lastMonth"	Filters by createdAt
page / pageSize	integer	≥ 1	Pagination, same defaults as the customer list (1 / 20)

Success response — 200

```
{
  "data": [
    {
      "id": "c8f2....",
      "subtotal": "323.50",
      "tax": "46.50",
      "serviceCharge": "10.00",
      "discount": "-23.50",
      "total": "356.50",
      "paymentMethod": "CASH",
      "invoiceNo": "INV-99871",
      "status": "PENDING",
      "createdAt": "2026-07-12T11:15:00.000Z",
      "device": { "name": "Register 1" },
      "lineItems": [
        {
          "id": "9a1c....",
          "name": "Wireless Mouse",
          "quantity": "1",
          "unitPrice": "75.00",
          "totalPrice": "75.00",
          "warrantyEndDate": "2027-07-12T00:00:00.000Z",
          "category": { "name": "Electronics" }
        }
      ]
    }
  ],
  "pagination": { "page": 1, "pageSize": 20, "total": 1, "totalPages": 1 }
}
```

Note what's different from the customer `/receipts` shape: no `device.store` block (the shop owner already knows it's their own store) — instead `device.name`, so a multi-register store can tell which till a sale came from.

GET /shop/receipts/:id

Same object shape as a `/shop/receipts` list item. `:id` is the receipt's UUID.

Status	Message	When
404	"Receipt not found"	<code>:id</code> doesn't exist
403	"This receipt does not belong to your store"	Receipt exists but was generated by a different store's device

GET /shop/dashboard

Success response — 200

```
{
  "storeName": "Monir Electronics",
  "totalRevenueThisMonth": "356.50",
  "receiptCountThisMonth": 1,
  "recentReceipts": [
    { "...same shape as a /shop/receipts list item...": true }
  ]
}
```

`totalRevenueThisMonth` is the plain number `0` (not a Decimal string) if the store has no receipts yet this month — same quirk noted in General Conventions above.

GET /shop/insights

Query parameters

Param	Values	Default
range	"thisMonth" "lastMonth" "3months"	"thisMonth" if omitted

Success response — 200

```
{
  "range": "thisMonth",
  "totalRevenue": "356.50",
  "categories": [
    { "category": "Electronics", "total": "75.00" },
    { "category": "Groceries", "total": "90.00" }
  ]
}
```

Same shape and sorting as the customer `/insights` , scoped to the store's total sales instead of one customer's spending.

Not Included In This Doc

The backend also has a `/devices` module (`POST /devices/pair/qr` , `POST /devices/pair/nfc` , `GET /devices/pair/status`) and two receipt endpoints (`POST /receipts` , `POST /receipts/:id/claim/qr`). These are deliberately **excluded** from this document because the customer mobile app never calls them:

- `POST /receipts` and `POST /receipts/:id/claim/qr` are called by the Raspberry Pi itself, authenticated with a per-device API key (`x-api-key` header), not a customer JWT.
- The `/devices/pair/*` endpoints are the one-time flow a shop owner uses to link a new Pi to their store — separate from viewing receipts/dashboard/insights, which is now documented above. No device-pairing screens have been shared yet, so that part stays excluded for now.

Security note (not blocking, but worth knowing): `GET /devices/pair/status` currently has **no authentication guard at all** — anyone who knows (or guesses) a device's `pairingId` can call it and receive that device's `apiKey` in plain JSON. This is the same class of issue as the earlier `device.apiKey` leak that was fixed on `/receipts`, just in a different endpoint. It's not part of today's build, flagging it here so it doesn't get forgotten.

Known Gaps / Open Items

1. Store branding fields are currently empty. `taxId` and `logoUrl` are `null` on the test store used for development. Set them in Prisma Studio before a demo that shows store branding on the Receipts/Details screens.

2. Smart Offers are mocked. The `smartOffers` array on `/dashboard` is hardcoded placeholder content — there is no real recommendation logic behind it.

3. Language and notification toggles are client-side only. Confirmed — no backend persistence, no endpoint to call for these Profile screen settings.